

Assignment: Building a Modular Data Sanitization & Exploration Engine

Background

In real-world data science, 80% of the work is spent cleaning and exploring data. Most of this work is repetitive: checking for nulls, identifying outliers, and visualizing distributions. Your task is to build a **Reusable Python Class** named `DataInspector` and a supporting `PlottingMethods` class that can be imported into Google Colab to automate these tasks.

The Objective

Develop an end-to-end tool for CSV data ingestion, advanced cleaning, feature engineering preparation, and high-level statistical visualization.

Technical Requirements

1. Data Ingestion & Sanitization

- **Colab Integration:** Implement `upload_data()` to handle local file uploads.
- **Garbage String Handling:** Automatically recognize and convert strings like `'?'`, `'n/a'`, `'NULL'`, and `' '` into actual `NaN` values.
- **Auto-Type Correction:** Force-convert columns to numeric types if the conversion does not result in an entirely null column.

2. Structural Analysis & Cleaning

- **Data Summary:** Provide a method to display row/column counts, a preview of the first 20 rows, and a breakdown of numerical vs. categorical columns.
- **Intelligent Imputation:** Create a `handle_missing_values()` method supporting multiple strategies: `mean`, `median`, `mode`, or a `constant` value.
- **Duplicate & Outlier Management:**
 - Implement `remove_duplicates()` to prune exact row matches.
 - Develop an **IQR-based** outlier detection system (`handle_outliers`) that allows users to flag or automatically delete rows based on specific columns.
- **Targeted Deletion:** Implement interactive methods (`delete_rows`, `delete_columns`) that accept comma-separated user input to prune the dataset.

3. Feature Engineering Preparation (Normalization)

- **Numeric Scaling:** Implement `extract_normalized_numeric_data()` supporting `minmax`, `standard` (Z-score), and `robust` (IQR-based) scaling.
- **Categorical Encoding:** Implement `extract_normalized_categorical_data()` supporting `onehot`, `ordinal`, and `uniform` (scaled 0-1) encoding.
- **Dataset Merging:** Provide a method to create a unified DataFrame containing original numeric data alongside encoded categorical data.

4. Advanced Interactive Visualization (Plotly)

- **Univariate Subplots:** For numeric columns, generate a 3-panel subplot: **Horizontal Violin/Box**, **Scatter Plot** (Index vs Value), and **Histogram**.
- **Smart Relationships:** Build a `plot_relationship()` tool that detects types and chooses the correct chart:
 - **Num-Num:** Scatter with OLS Trendline.
 - **Cat-Num:** Box plot with all data points.
 - **Cat-Cat:** Grouped Bar chart.
- **Categorical Frequency:** Create bar charts displaying both raw counts and percentage labels.

5. Deep Statistical Insights

- **Unified Heatmap:** Develop `plot_all_associations_heatmap()` to visualize relationships across *all* data types:
 - **Numeric-Numeric:** Pearson's r .
 - **Categorical-Categorical:** Cramér's V .
 - **Mixed (Num-Cat):** Point-Biserial correlation or Eta (via ANOVA).

6. Custom Modular Plotting

Implement a separate `PlottingMethods` class to handle granular chart generation (Bar, Pie, Histogram) that returns HTML-wrapped figures for flexible embedding.

Submission Criteria

1. **Object-Oriented Design:** All logic must be encapsulated within the `DataInspector` and `PlottingMethods` classes.
2. **Clean Code:** Every method must include descriptive **Docstrings** and handle empty/None data gracefully.
3. **Real-world Testing:** Demonstrate the tool using a dataset (e.g., Titanic) by performing a full flow: Upload → Impute → Normalize → Visualize Associations.

